

7.2 - Error budget

O saldo que decide entre acelerar e estabilizar

Error budget transforma o SLO em decisão

De objetivo passivo a ferramenta ativa

- **Error budget é o complemento direto do seu SLO**
 - se o SLO é 99,9%, o budget são os 0,1% restantes
- **Ele mede quanta falha ainda é tolerável**
 - cada incidente consome uma fatia desse saldo mensal
- **Budget cheio libera deploys e experimentos de risco**
 - há margem para errar sem quebrar o compromisso
- **Budget quase zerado obriga foco em estabilidade**
 - congela mudanças arriscadas até o saldo se recuperar

O que o saldo do budget determina

Budget cheio

- Deploys frequentes são liberados
- Experimentos de risco são aceitáveis
- Time prioriza velocidade de entrega
- Espaço para inovar sem medo

Budget quase zerado

- Mudanças arriscadas são congeladas
- Foco migra para estabilização
- Postmortems viram prioridade máxima
- Velocidade cede lugar à confiabilidade

O saldo decide o ritmo do mês

- **SLOs definidos, error budget usado como decisão**
 - mês de budget cheio contra mês de budget consumido

LUNALOG

Com os SLOs definidos, a LunaLog passou a usar o error budget como mecanismo de decisão objetivo. Num mês em que o budget estava cheio, o time liberou três deploys de alto risco com features novas. Em outro mês, dois incidentes consumiram 60% do budget e o time congelou mudanças arriscadas para focar em estabilização. A pergunta 'será que é seguro fazer esse deploy?' deixou de ser subjetiva e passou a ter resposta no número.

O budget alinha engenharia e negócio

- **Sem budget, a decisão de arriscar vira opinião**
- **Com budget, vira política combinada com antecedência**

O error budget tira a discussão de deploy do campo do achismo. A regra é definida antes do incidente, não no calor dele: enquanto há saldo, acelera-se; quando o saldo acaba, estabiliza-se. Negócio e engenharia concordam com a regra uma vez, e ela passa a governar sozinha depois.

- Apresente o budget como linguagem de risco para a liderança



O error budget governa quando você pode arriscar. Mas há uma situação que nenhum saldo previne sozinho: o momento em que um serviço chama outro e a resposta não chega. A dependência travou. E aí, o que o seu sistema faz? Se ele insistir do jeito errado, uma falha pequena numa ponta vira um colapso em cadeia. Existem três padrões que separam quem absorve a falha de quem a amplifica.

Retry, timeout e backoff exponencial